Architectures logicielles et matérielles

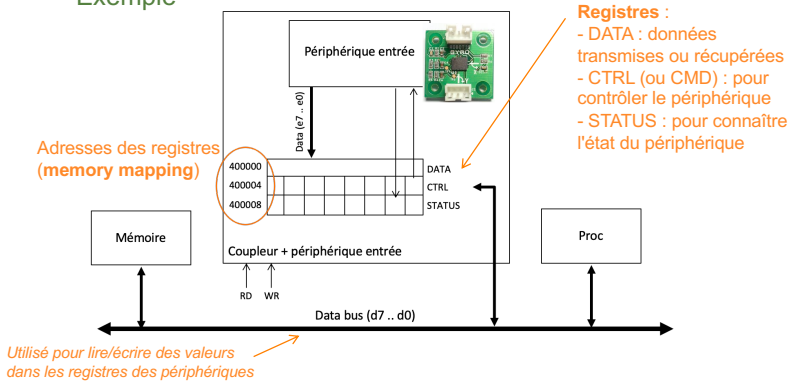
Cours 4 : Communications via polling/interruptions

Laurence Pierre

UGA Université Grenoble Alpes L3 Informatique 2026-2027

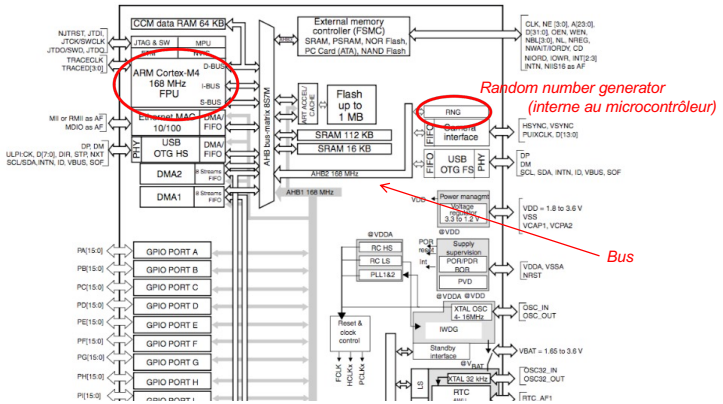
Rappels périphériques

- Le processeur est relié aux périphériques via des coupleurs (contrôleurs) d'entrées/sorties
  - Exemple



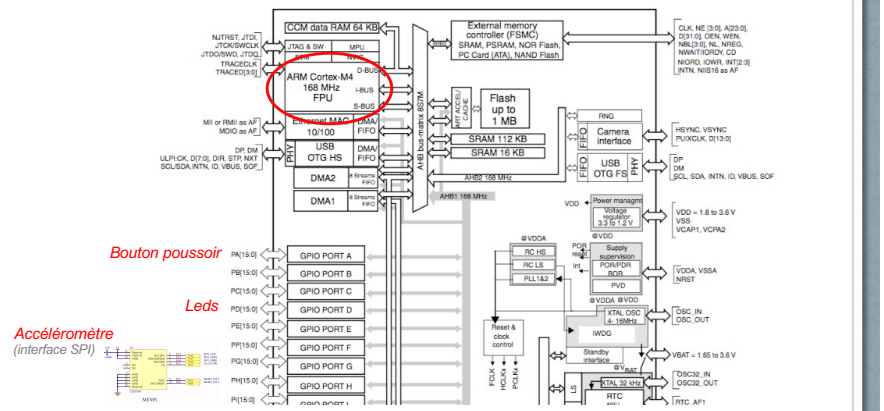
Rappels périphériques

- Exemple : Cas du RNG (vu en TP)



Rappels périphériques

- Pour les périphériques externes, utilisation des ports GPIO voire autres interfaces (SPI, I2C) - voir plus tard

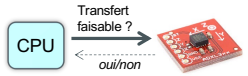


Transfert de données

- Les **transferts de données** entre le processeur et le périphérique (ex. valeurs transmises par un capteur) se font ***lorsque l'état du périphérique le permet*** (données valides disponibles, ou périphérique libre pour recevoir,...)

- Comment gère-t-on cela ?

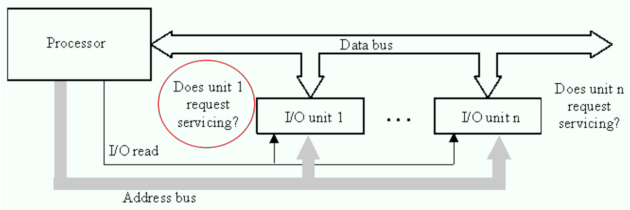
1. Solution avec **polling** (attente active)
   - Contrôlé par logiciel
2. Ou solution avec utilisation d'**interruptions**





1 Polling

- Dans le cas du **polling** (attente active), le **logiciel interroge** le périphérique répétitivement pour savoir *si son état indique qu'il est prêt* (**bits dédiés** dans le registre d'état, selon les périphériques)

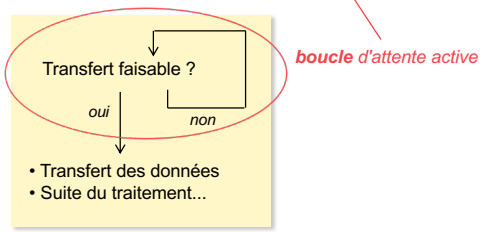


https://edux.pjwstk.edu.pl/mat/264/lec/main99.html

Polling

- Dans le cas du **polling** (attente active), le **logiciel interroge** le périphérique répétitivement pour savoir *si son état indique qu'il est prêt* (**bits dédiés** dans le registre d'état, selon les périphériques)

- Le transfert (et la suite du traitement) ne sera réalisé qu'en sortie de cette attente

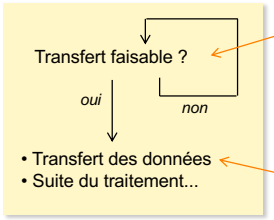


***boucle** d'attente active*

Polling

- Dans le cas du **polling** (attente active), le **logiciel interroge** le périphérique répétitivement pour savoir *si son état indique qu'il est prêt* (**bits dédiés** dans le registre d'état, selon les périphériques)

- Le transfert (et la suite du traitement) ne sera réalisé qu'en sortie de cette attente



Inspecter l'état du périphérique => lecture du registre STATUS (via bus data), qui est mis à jour par le périphérique

***Masquage** nécessaire si certains bits doivent être sélectionnés*

Transfert avec registre DATA - lecture ou écriture (via bus data)

Polling

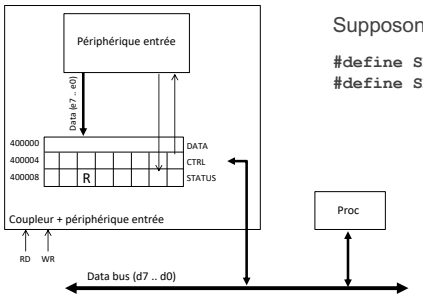
- Dans le cas du **polling** (attente active), le **logiciel interroge** le périphérique répétitivement pour savoir *si son état indique qu'il est prêt* (**bits dédiés** dans le registre d'état, selon les périphériques)
- Le transfert (et la suite du traitement) ne sera réalisé qu'en sortie de cette attente

```
...
Tant que (attente nécessaire)
   inspecter état du périphérique;
Fin tant que
Faire action voulue, par ex. transférer
et traiter donnée;
...
```

L'implémentation dépend des caractéristiques du périphérique (et de la bibliothèque utilisée)

Polling - Consultation état

- Informations sur l'état du périphérique (ex. prêt pour communication) ?
  - → bits dédiés dans le **registre d'état**
- Rappels - Exemple



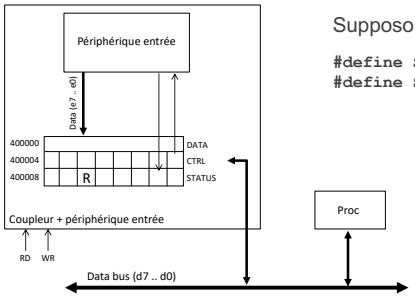
Supposons que soient donnés :

```
#define SR (*(volatile uint8_t *)(0x400008))
#define SR_R (1 << 5)
```

Que représentent ces macros ?

Polling - Consultation état

- Informations sur l'état du périphérique (ex. prêt pour communication) ?
  - → bits dédiés dans le **registre d'état**
- Rappels - Exemple



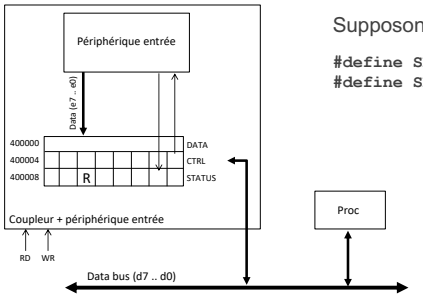
Supposons que soient donnés :

```
#define SR (*(volatile uint8_t *)(0x400008))
#define SR_R (1 << 5)
```

Comment **observer** uniquement le bit R du registre ?

Polling - Consultation état

- Informations sur l'état du périphérique (ex. prêt pour communication) ?
  - → bits dédiés dans le **registre d'état**
- Rappels - Exemple



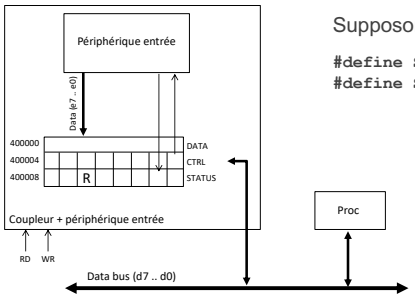
Supposons que soient donnés :

```
#define SR (*(volatile uint8_t *)(0x400008))
#define SR_R (1 << 5)
```

Quelle condition écrire pour savoir si le bit R du registre **est à 1** ?

Polling - Consultation état

- Informations sur l'état du périphérique (ex. prêt pour communication) ?
  - → bits dédiés dans le **registre d'état**
- Rappels - Exemple



Supposons que soient donnés :

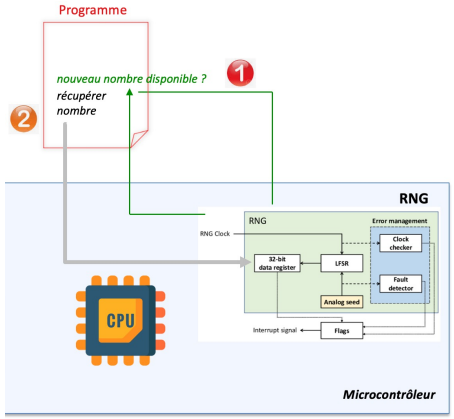
```
#define SR (*(volatile uint8_t *)(0x400008))
#define SR_R (1 << 5)
```

En **exercice** à préparer : comment écrire (en C) la **boucle de polling** pour attendre que le bit R du registre STATUS soit passé à 1 ?

Polling - Cas du RNG

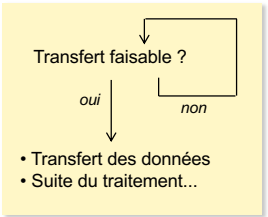
- En TP : **récupération d'un nombre** aléatoire (registre de données), chaque fois qu'un nouveau nombre est disponible (registre de status)

```
Tant que (nouveau nombre pas disponible);
Récupérer nombre;
```



Polling - Réflexion

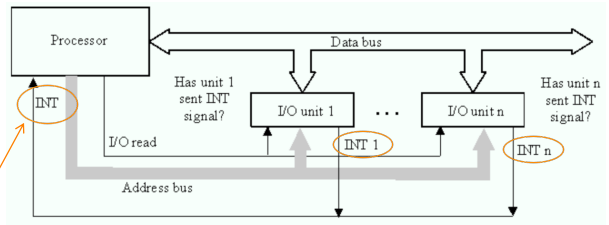
- Dans le cas du **polling** (attente active), le **logiciel interroge** le périphérique répétitivement pour savoir *si son état indique qu'il est prêt* (bits dédiés dans le registre d'état, selon les périphériques)
- Le transfert (et la suite du traitement) ne sera réalisé qu'en sortie de cette attente



Mais voyez-vous des inconvénients à une telle solution ?

2 Utilisation d'interruptions

- Dans ce cas, le **périphérique** sera chargé de "prévenir" lui-même *lorsqu'il est prêt* pour la communication ⇒ envoi d'une "**interruption** matérielle" (IRQ)



https://edux.pjwstk.edu.pl/mat/264/lec/main99.html

Le processeur prend en compte une interruption provenant d'un des périphériques, il gère le déroutement sur un handler (et le retour d'interruption) -> Voir cours 5

*Pour que cela soit possible, il est nécessaire de **configurer pour autoriser les interruptions**, à plusieurs niveaux (périphérique, processeur)...*

Utilisation d'interruptions

- Dans ce cas, le **périphérique** sera chargé de "prévenir" lui-même *lorsqu'il est prêt* pour la communication ⇒ envoi d'une "**interruption** matérielle" (IRQ)
- La réception de cette interruption devra provoquer un **déroutement vers un handler** (gestionnaire) de l'interruption, chargé de réaliser le transfert (voire un traitement)

• Configuration du périphérique pour utilisation d'interruption
• ...
• Boucle de traitement normal

Utilisation d'interruptions

- Dans ce cas, le **périphérique** sera chargé de "prévenir" lui-même *lorsqu'il est prêt* pour la communication ⇒ envoi d'une "**interruption** matérielle" (IRQ)
- La réception de cette interruption devra provoquer un **déroutement vers un handler** (gestionnaire) de l'interruption, chargé de réaliser le transfert (voire un traitement)



• Configuration du périphérique pour utilisation d'interruption
• ...
• Boucle de traitement normal

Traitant d'interruption (handler)
• Transfert des données
• Annulation interruption

Utilisation d'interruptions

- Dans ce cas, le **périphérique** sera chargé de "prévenir" lui-même *lorsqu'il est prêt* pour la communication ⇒ envoi d'une "**interruption** matérielle" (IRQ)
- La réception de cette interruption devra provoquer un **déroutement vers un handler** (gestionnaire) de l'interruption, chargé de réaliser le transfert (voire un traitement)



• Configuration du périphérique pour utilisation d'interruption
• ...
• Boucle de traitement normal

Traitant d'interruption (handler)
• Transfert des données
• Annulation interruption

retour d'interruption

Utilisation d'interruptions

- Dans ce cas, le **périphérique** sera chargé de "prévenir" lui-même *lorsqu'il est prêt* pour la communication ⇒ envoi d'une "**interruption** matérielle" (IRQ)
- La réception de cette interruption devra provoquer un **déroutement vers un handler** (gestionnaire) de l'interruption, chargé de réaliser le transfert (voire un traitement)

Transfert avec registre DATA - lecture ou écriture (via bus data)



• Configuration du périphérique pour utilisation d'interruption
• ...
• Boucle de traitement normal

Traitant d'interruption (handler)
• Transfert des données
• Annulation interruption

retour d'interruption

*Et nous verrons au cours 5 comment le **processeur** gère l'interruption (déroutement sur le handler) et le retour d'interruption*

En général l'interruption doit être annulée (acquittée)

Utilisation d'interruptions

- Dans ce cas, le **périphérique** sera chargé de "prévenir" lui-même *lorsqu'il est prêt* pour la communication ⇒ envoi d'une "**interruption** matérielle" (IRQ)
- La réception de cette interruption devra provoquer un **déroutement vers un handler** (gestionnaire) de l'interruption

```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it;
Activer interruption du périphérique;
Faire traitement normal;
```

(NB. l'interrupt. peut être autorisée par défaut)

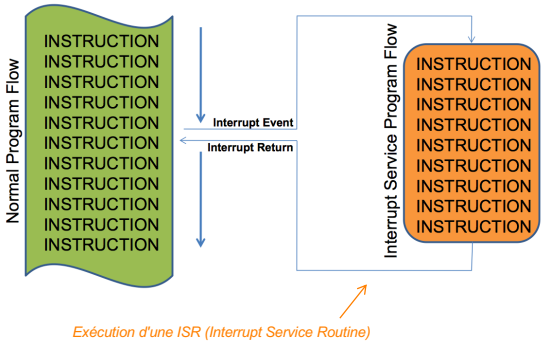
(NB. peut faire l'objet d'une association prédéfinie)

```
Fonction gest_it :
    Faire l'action associée à
    l'interruption...
    Annuler (acquitter) interruption;
```

L'implémentation dépend des caractéristiques du périphérique (et de la bibliothèque utilisée)

Gestion des interruptions

- C'est le rôle du contrôleur d'interruption de veiller à transmettre correctement l'interruption, pour interrompre le flot normal d'exécution

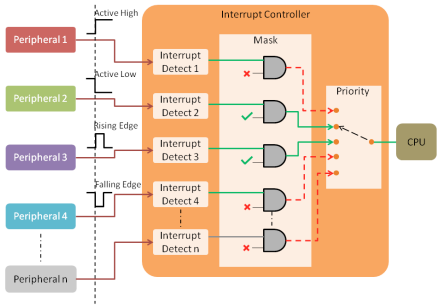


Exécution d'une ISR (Interrupt Service Routine)

http://slideplayer.com/slide/6991909/

Gestion des interruptions

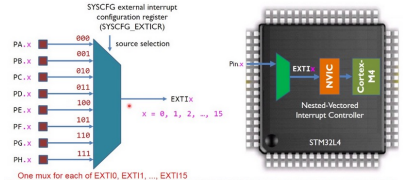
- Le contrôleur d'interruption enregistre les demandes d'interruption, gère les masquages et priorités, signale au processeur la requête d'interruption et lui transmet les informations nécessaires



http://embien.com/blog/interrupt-handling-in-embedded-software/

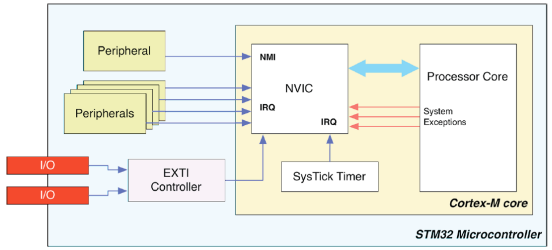
Gestion des interruptions

- Une interruption matérielle sera mémorisée dans un registre de requêtes en attente, et
  - ne sera prise en compte que si le registre de **masquage** des interruptions a été configuré pour le permettre
  - lorsque l'interruption aura été traitée et **acquittée**, le bit correspondant du registre des requêtes sera remis à 0
- Interruptions extérieures => les signaux arrivant via les GPIO pourront être repérés via leurs fronts montants, descendants ou les deux, selon la configuration choisie pour le EXTI



Gestion des interruptions

- Sur STM32F4, gestion des interruptions faite par un NVIC (Nested Vectored Interrupt Controller)

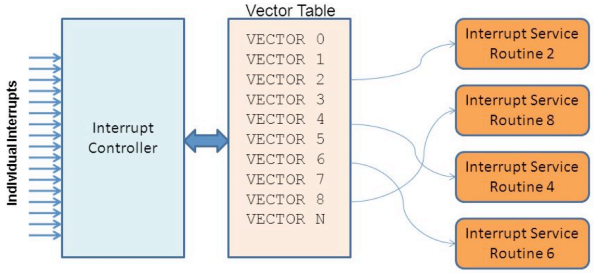


http://blog.naver.com/PostView.nhn?blogId=gauya&logNo=220974576065

  - NB. Il peut prendre en compte les interruptions matérielles venant des périphériques (IRQ), et les interruptions logicielles (exceptions)

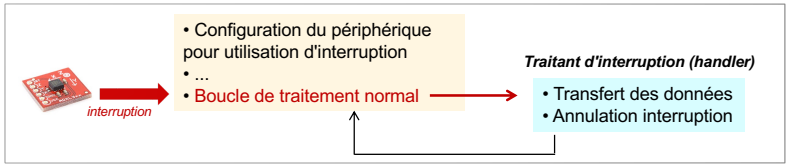
Gestion des interruptions

- La gestion des interruptions utilise une table des **vecteurs d'interruptions** (IVT, interrupt vector table), zone de mémoire qui fournit les **adresses des handlers** associés au traitement des interruptions (**ISR**, Interrupt Service Routine)



Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**




```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it;
Activer interruption du périphérique;
Faire traitement normal;
```

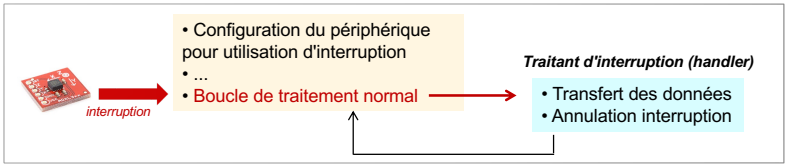
?

```
Fonction gest_it :
   Faire l'action associée à
   l'interruption...
   Annuler (acquitter) interruption;
```

?

Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**




```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it;
Activer interruption du périphérique;
Faire traitement normal;
```

Activation interruption *irqn* (dans le registre de masquage) :

```
void nvic_enable_irq (uint8_t irqn);
```

```
Fonction gest_it :
   Faire l'action associée à
   l'interruption...
   Annuler (acquitter) interruption;
```

Par exemple :

```
nvic_clear_pending_irq(...);
```

Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**

```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it;
Activer interruption du périphérique;
Faire traitement normal;
```

Activation interruption *irqn* (dans le registre de masquage)

Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**

```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it;
Activer interruption du périphérique;
Faire traitement normal;
```

Activation interruption *irqn* (dans le registre de masquage) :

```
void nvic_enable_irq (uint8_t irqn);
```

Fichier nvic.h

```
#define NVIC_NVIC_WWDG_IRQ 0
#define NVIC_PVD_IRQ 1
#define NVIC_TAMP_STAMP_IRQ 2
#define NVIC_RTC_WKUP_IRQ 3
#define NVIC_FLASH_IRQ 4
#define NVIC_RCC_IRQ 5
...
#define NVIC_TIM4_IRQ 30
...
#define NVIC_HASH_RNG_IRQ 80
...
```


Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**

```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it;
Activer interruption du périphérique;
Faire traitement normal;
```

Activation interruption *irqn* (dans le registre de masquage) :

```
void nvic_enable_irq (uint8_t irqn);
```

Exemples :

Appel pour le timer4 ?
Appel pour le RNG ? ?

Fichier nvic.h

```
#define NVIC_NVIC_WWDG_IRQ 0
#define NVIC_PVD_IRQ 1
#define NVIC_TAMP_STAMP_IRQ 2
#define NVIC_RTC_WKUP_IRQ 3
#define NVIC_FLASH_IRQ 4
#define NVIC_RCC_IRQ 5
...
#define NVIC_TIM4_IRQ 30
...
#define NVIC_HASH_RNG_IRQ 80
...
```


Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**

```
Fonction gest_it :
   Faire l'action associée à
   l'interruption...
   Annuler (acquitter) interruption;
```

La gestion des interruptions utilise une **table des "vecteurs d'interruptions"**, zone de mémoire qui contient les adresses des ISR

Fichier vector_nvic.c

```
#define IRQ_HANDLERS \
   [NVIC_NVIC_WWDG_IRQ] = nvic_wwdg_isr, \
   [NVIC_PVD_IRQ] = pvd_isr, \
   [NVIC_TAMP_STAMP_IRQ] = tamp_stamp_isr, \
   [NVIC_RTC_WKUP_IRQ] = rtc_wkup_isr, \
   ...
   [NVIC_TIM4_IRQ] = tim4_isr, \
   ...
   [NVIC_HASH_RNG_IRQ] = hash_rng_isr, \
   ...
   [NVIC_DMA2D_IRQ] = dma2d_isr
```

Configuration des interruptions

- Sur STM32F4, avec utilisation de la **bibliothèque libopencm3**

```
Fonction gest_it :
   Faire l'action associée à
   l'interruption...
   Annuler (acquitter) interruption;
```

La gestion des interruptions utilise une **table des "vecteurs d'interruptions"**, zone de mémoire qui contient les adresses des ISR

Exemples :

ISR pour le timer4 ?
ISR pour le RNG ? ?

Fichier vector_nvic.c

```
#define IRQ_HANDLERS \
   [NVIC_NVIC_WWDG_IRQ] = nvic_wwdg_isr, \
   [NVIC_PVD_IRQ] = pvd_isr, \
   [NVIC_TAMP_STAMP_IRQ] = tamp_stamp_isr, \
   [NVIC_RTC_WKUP_IRQ] = rtc_wkup_isr, \
   ...
   [NVIC_TIM4_IRQ] = tim4_isr, \
   ...
   [NVIC_HASH_RNG_IRQ] = hash_rng_isr, \
   ...
   [NVIC_DMA2D_IRQ] = dma2d_isr
```


Configuration des interruptions

- Exemple/exercice : globalement, que deviendrait ce pseudo-code **pour le timer4** ?

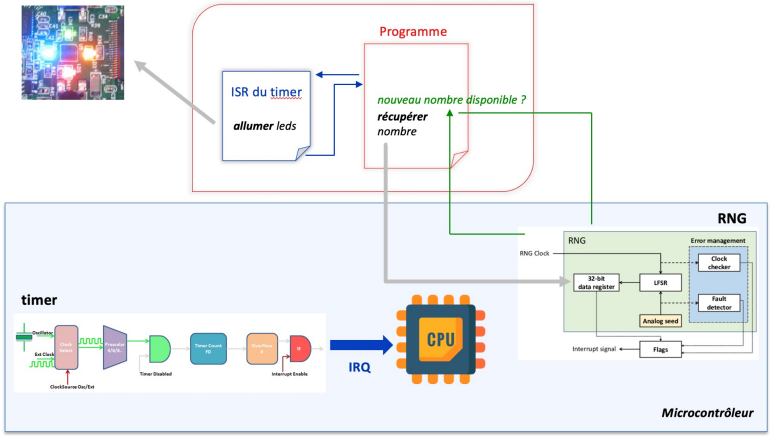
```
Autoriser (au niveau du proc) l'interruption
venant du périphérique;
Associer interruption à fonction gest_it; (NB. peut faire l'objet d'une association prédéfinie)
Activer interruption du périphérique;
Faire traitement normal;
```

```
Fonction gest_it :
   Faire l'action associée à
   l'interruption...
   Annuler (acquitter) interruption;
```

→ ?

TP RNG - Utilisation interrup. du timer

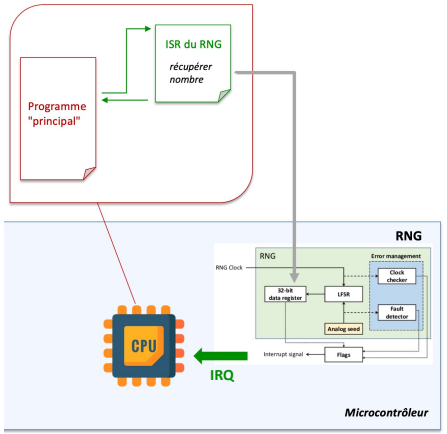
- Et timer pour **allumage** périodique **des leds** (cf slide 14)



TP RNG version 2

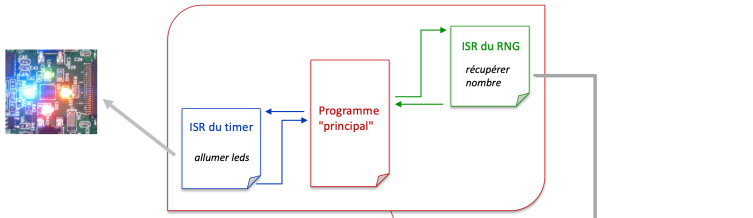
- En TP, version 2 pour la **récupération du nombre** : émission d'une interruption chaque fois qu'un nouveau nombre est disponible
  => alors, récupération d'un nombre aléatoire (registre de données)

Bien observer la différence avec le slide 14 !



TP RNG version 2

- Et ajout timer pour **allumage** périodique **des leds**



Bien observer ce qui est commun avec le slide 38 !

